

Manuale Tecnico TheKnife

Autori:

- Youssef – Project Manager, Document & Quality Manager
- Federico Dellatorre – Feature Developer, Debugging & Documentation

Indice

1. Struttura dell'applicazione
2. Scelte Architettureali
3. Strutture dati
4. Algoritmi e logica
5. Sicurezza e validazione
6. Formato dei file e Gestione dei path
7. Debug e test
8. Uso di pattern
9. Estensioni future
10. Conclusione

1. Struttura dell'applicazione

L'applicazione è suddivisa in moduli con responsabilità ben definite secondo il principio **Single Responsibility**:

-Main.java: entry point dell'app

-TheKnife.java: gestione dei menù, interfacce e flusso utente

-GestioneUtenti.java: registrazione, login, gestione utenti

-RecensioneManager.java: gestione recensioni e file associati

-RistoranteManager.java: caricamento da CSV, ricerca e gestione ristoranti

-Utente.java:

Dati anagrafici: nome, cognome, domicilio, dataDiNascita

Credenziali: username, passwordCifrata

Ruolo: isRistoratore → determina i permessi nel menu

Preferiti: List<Ristorante> → lista dinamica dei ristoranti salvati

-Recensione.java: Modella una valutazione lasciata da un utente su un ristorante.

Campi principali:

- **autore:** riferimento all'utente (via username)
- **nomeRistorante:** per collegamento logico alla destinazione
- **testo, stelle, data:** contenuto valutativo
- **rispostaDelRistoratore:** opzionale, inserita post-recensione

-Ristorante.java: È la classe più ricca tra le entità, descrive ogni attività gastronomica presente nel sistema.

Campi principali:

Dati descrittivi: nome, indirizzo, città, nazione, tipoCucina

Servizi offerti: deliveryDisponibile, prenotazioneOnlineDisponibile

Coordinate GPS: latitudine, longitudine

Fascia economica: fasciaPrezzo, prezzoMedio

Recensioni: List<Recensione> → collegate dinamicamente

2. Scelte Architettureali

1) Modularità: ogni classe gestisce un ambito specifico

2) Separazione tra logica e dati: i manager gestiscono la logica, i modelli rappresentano i dati

3) Estendibilità: nuove funzionalità (es. prenotazioni) possono essere aggiunte senza modificare la struttura esistente

L'app TheKnife è stata progettata con una struttura a moduli separati, dove ogni classe ha una responsabilità precisa:

Motivazioni delle scelte

Abbiamo optato per una struttura modulare per:

Classe	Scopo
Theknife	Controlla il flusso generale del programma (menu, ruoli, sessione)
GestioneUtenti	Gestisce registrazione, login e salvataggio degli utenti
RistoranteManager	Si occupa della ricerca, aggiunta e caricamento dei ristoranti
RecensioneManager	Gestisce le recensioni (creazione, modifica, salvataggio)
Utente	Modello utente: dati, credenziali, ruolo, preferiti
Ristorante	Modello ristorante: descrizione, servizi, coordinate, recensioni
Recensione	Modello recensione: autore, testo, stelle, data, risposta

- **Separare le logiche:** ogni classe si occupa solo di un ambito (es. login, recensioni, ricerca)
- **Rendere il codice più leggibile e mantenibile**
- **Facilitare test e debug** su singole parti

Per implementare le funzioni di ricerca avanzata abbiamo usato lo **Stream API di Java**, che consente di applicare filtri multipli in modo leggibile e funzionale:

```
ristoranti.stream()
```

```
.filter(r -> r.getPrezzoMedio() <= prezzoMax)
```

```
.filter(r -> r.getMediaStelle() >= minStelle)
```

...

3. Strutture Dati Utilizzate

Nel progetto TheKnife abbiamo utilizzato principalmente **liste** (ArrayList) per gestire e raccogliere dati dinamici, senza complicare la struttura con database o collezioni avanzate.

Liste principali:

- List<Utente> → mantiene tutti gli utenti registrati nel sistema.
- List<Ristorante> → contiene tutti i ristoranti caricati da CSV o aggiunti dai ristoratori.
- List<Recensione> → raccoglie le recensioni inserite dagli utenti.
- List<Ristorante> nei preferiti (Utente) → ogni utente può memorizzare i ristoranti preferiti in una lista personale.
- List<Recensione> nel singolo Ristorante → ogni ristorante tiene traccia delle recensioni ricevute.

Motivazione della scelta

Abbiamo scelto le ArrayList perché:

- Sono **facili da usare** e leggere/modificare
- Offrono **accesso diretto** tramite indice
- Sono ideali per applicazioni **console** e progetti didattici
- Supportano bene le operazioni di ricerca con Java Stream

4. Scelte Algoritmiche

L'app TheKnife utilizza algoritmi semplici ma efficaci per eseguire operazioni come ricerca, filtro, media e validazione. Abbiamo scelto soluzioni lineari e funzionali per mantenere il codice leggibile e veloce da testare.

Ricerca e Filtri

Per cercare ristoranti in base a criteri come città, tipo cucina, prezzo o media stelle, usiamo l'**API Stream di Java**, che rende il codice molto compatto:

```
ristoranti.stream()
```

```
.filter(r -> r.getCitta().equalsIgnoreCase(citta))
```

```
.filter(r -> r.getPrezzoMedio() <= prezzoMax)
```

```
.filter(r -> r.getMediaStelle() >= minStelle)
```

```
.collect(Collectors.toList());
```

Questo approccio è più elegante rispetto ai classici cicli for, e permette di **combinare più condizioni** in modo chiaro.

Calcolo media recensioni

Per ogni ristorante, viene calcolata la media delle stelle ricevute:

```
public double getMediaStelle() {  
    if (recensioni.isEmpty()) return 0;  
  
    double somma = 0;  
  
    for (Recensione r : recensioni) {  
        somma += r.getStelle();  
    }  
  
    return somma / recensioni.size();  
}
```

La media è usata anche nei filtri per trovare ristoranti ben recensiti.

Abbiamo implementato un metodo `cercaCombinata(...)` che **unisce più criteri di ricerca** (es. città, cucina, prezzo, servizi, media):

```
.filter(r -> !requireDelivery || r.isDeliveryDisponibile())
```

Qui l'algoritmo controlla che il ristorante soddisfi solo i criteri **che l'utente ha scelto**. Questo evita filtri rigidi e permette ricerche flessibili.

Validazioni input

Le conversioni numeriche (int, double) sono protette da try/catch, e le scelte utente sono sempre controllate per evitare crash o errori:

```
try {  
  
    int valore = Integer.parseInt(scanner.nextLine());
```

```
} catch (NumberFormatException e) {  
    System.out.println("Formato non valido.");  
}
```

5. Sicurezza e Validazione

- Cifratura password

```
MessageDigest md = MessageDigest.getInstance("SHA-256");  
  
byte[] hash = md.digest(password.getBytes(StandardCharsets.UTF_8));  
  
String digest = bytesToHex(hash);
```

- Password mai salvata in chiaro
- Digest irreversibile → sicurezza garantita

- Validazione input

```
try {  
    int scelta = Integer.parseInt(scanner.nextLine());  
} catch (NumberFormatException e) {  
    System.out.println("Formato non valido.");  
}
```

- Protegge da crash
- Gestione robusta degli errori utente

6. Formato dei file e Gestione dei Path

Nel progetto TheKnife vengono usati due formati di file:

1. File binari (.dat)

Utilizzati per salvare i dati dell'applicazione durante l'uso:

- data/utenti.dat → utenti registrati
- data/recensioni.dat → tutte le recensioni

- data/ristoranti.dat → ristoranti gestiti nel programma

I dati vengono salvati usando la serializzazione (ObjectOutputStream) e letti con ObjectInputStream. Questa scelta semplifica la gestione e mantiene il contenuto compatto.

2. File CSV (.csv)

Usato solo all'avvio per importare i ristoranti Michelin:

- data/michelin_my_maps.csv → contiene i dati iniziali dei ristoranti (nome, tipo cucina, coordinate, ecc.)

Viene letto una sola volta in caso di prima esecuzione o se il file binario è vuoto.

Gestione dei percorsi (path)

Tutti i file sono conservati nella cartella data/, e vengono gestiti tramite **path relativi**, ad esempio:

```
java
```

```
gestioneUtenti.salvaSuFile("data/utenti.dat");
```

Questo garantisce:

- Portabilità del progetto su qualsiasi computer
- Evita problemi di permessi o posizioni assolute
- Funziona anche in ambienti IDE come IntelliJ, Eclipse, VS Code

7. Debug e Test

Messaggi di errore chiari: `System.out.println("Errore salvataggio utenti: " + e.getMessage());`

Controllo file mancanti → generazione automatica di liste vuote

Test manuali su:

- Login
- Registrazione
- Ricerca ristoranti
- Inserimento recensioni
- Risposte del ristorante

8. Uso di Pattern

- Non sono stati usati design pattern formali
- Il progetto segue:
 - **Modularità**
 - **Responsabilità singola**
 - **Separazione tra dati e logica**
 - **Gestione robusta degli errori**
 - **Uso di Stream API per codice funzionale**

9. Estensioni Future

- Interfaccia grafica con JavaFX
- Database relazionale (es. SQLite) al posto dei file .dat
- Sistema di prenotazioni
- Ordinamento per stelle, prezzo, distanza
- Notifiche per nuove recensioni
- Login con sessione e token

10. Conclusione

Il progetto *TheKnife* nasce dall'idea di creare un'applicazione semplice ma funzionale, capace di offrire un'esperienza utente chiara e completa nella gestione e valutazione dei ristoranti. Durante lo sviluppo, abbiamo cercato di bilanciare l'efficienza del codice con la leggibilità, adottando soluzioni modulari e sfruttando le potenzialità offerte da Java, come la Stream API e la serializzazione.

Ogni componente è stato pensato per essere estendibile e facilmente manutenibile, con particolare attenzione alla separazione delle responsabilità e alla gestione sicura dei dati. Pur trattandosi di un progetto didattico, abbiamo voluto simulare dinamiche reali, come la gestione dei ruoli, la persistenza dei dati e l'interazione tra utenti e contenuti.

Siamo consapevoli che ci siano margini di miglioramento — dall'interfaccia grafica all'integrazione con database — ma riteniamo che *TheKnife* rappresenti una base solida su cui iniziare a costruire.

